Homotopical Encrypt: The Harmonic Cochain Problem and Certified Public Shadows

Chloe Vander Vlugt

Abstract

We introduce the Harmonic Cochain Problem (HCP), a hidden-operator recovery problem in which public cochain or class-level data does not determine the operative Hodge–Laplacian kernel. In the Homotopical Encrypt model, public samples have the form $b = h + e$, where $h \in \ker(\Delta)$ is a harmonic cochain and $e \in \mathrm{im}(\delta) \oplus \mathrm{im}(\delta^*)$ is Hodge-structured noise. We compare HCP with the closest vector problem (CVP): CVP is the known-lattice comparison problem, while HCP hides the operator and kernel that make the relevant lattice explicit. We also describe a certified projection chain from accepted hidden/geometric state to a finite Reed–Solomon mutual-correlated-agreement target, with theorem boundaries formalized in Lean.

Contents

1 Introduction **3**
- 1.1 Motivation . . . 3
- 1.2 Contributions . . . 3

2 The Harmonic Cochain Problem **3**
- 2.1 Public And Hidden Data . . . 4
- 2.2 Recovery Target . . . 4
- 2.3 Computational Model . . . 4

3 Mathematical Framework **4**
- 3.1 Fibrational Semantics . . . 4
- 3.2 Fibrational Identity Objects . . . 5
- 3.3 Homotopy Classes . . . 5

4 Cochain Commitment Framework **5**
4.1 Cochain Commitments 5
4.2 Commitment Map 5
4.3 Projection Operator 5
4.4 Fundamental Properties 5

5 Projection Framework **6**
5.1 Projection Theorem 6
5.2 Preserved Structure 6
5.3 Lost Structure 6

6 Relationship To CVP **6**
6.1 Reduction Direction 6
6.2 Oracle Insufficiency 7
6.3 Structural Comparison 7

7 Formal Verification **7**
7.1 Lean Formalization 7
7.2 Repository Structure 7
7.3 Theorem Correspondence 7

8 Reed–Solomon Projection And Proximity Corollary **8**
8.1 Canonical Reed–Solomon Endpoint 8
8.2 Mutual Correlated Agreement 8
8.3 Corollary 8

9 Future Directions **8**
9.1 Cryptographic Protocol 8
9.2 Anonymous Identity System 8
9.3 Sheaf Storage 8
9.4 Vertra Chain 8
9.5 Complexity-Theoretic Extensions 9

A Repository Dependency Graph **9**

B Lean/Paper Correspondence **9**

1 Introduction

1.1 Motivation

Homotopical Encrypt is organized around the following security shape:

$$\text{public cochain/shadow data} \quad \not\Rightarrow \quad \text{the operative Hodge–Laplacian kernel.}$$

The private recovery object is not merely a scalar secret or a vector in a published lattice. It is the harmonic cochain selected by a hidden Hodge operator. Public verification therefore needs finite artifacts, while security must not collapse to recovering the private representative or the hidden operator.

1.2 Contributions

This paper develops the following contributions.

(i) The Harmonic Cochain Problem (HCP), a hidden-operator recovery problem for Hodge-theoretic cochains.

(ii) A Hodge-structured noise model with public samples $b = h + e$, where $h \in \ker(\Delta)$ and $e \in \operatorname{im}(\delta) \oplus \operatorname{im}(\delta^*)$.

(iii) A comparison with CVP, separating known-lattice closest-vector search from hidden-operator harmonic-cochain recovery.

(iv) A projection framework from hidden/geometric state to public finite evidence.

(v) A Lean formalization of the definitions, reductions, projection chain, gates, and theorem boundaries.

(vi) A Reed–Solomon mutual-correlated-agreement corollary aligned with the vocabulary of the Proximity Prize problem statement.

2 The Harmonic Cochain Problem

This section states the central search problem and the hidden-operator model in which it is used.

Definition 2.1 (Harmonic Cochain Problem)**.** *The* Harmonic Cochain Problem *(HCP) asks, given public cochain/class-level data associated to a Hodge system, to recover the harmonic cochain determined by the operative Laplacian kernel.*

In the Homotopical Encrypt public-sample model, the visible cochain has the form

$$b = h + e, \qquad h \in \ker(\Delta), \qquad e \in \operatorname{im}(\delta) \oplus \operatorname{im}(\delta^*).$$

Here h is the harmonic cochain, equivalently the global-section component, and e is Hodge-structured noise. The hidden-operator condition is that the public data does not determine a unique Δ or $\ker(\Delta)$.

2.1 Public And Hidden Data

[TODO: State the public interface: cochain/class-level data, public shadows, commitments, topology hashes, oracle judgments, and proof-carrying artifacts.]

[TODO: State the hidden interface: fibration representative, restriction maps, Laplacian, harmonic kernel, and private witness/cochain data.]

2.2 Recovery Target

The recovery target is the harmonic cochain selected by the operative kernel, not necessarily the entire kernel subspace. This distinction keeps HCP parallel to CVP: CVP asks for a closest vector in a presented lattice, while HCP asks for a harmonic cochain in the operative Hodge kernel.

2.3 Computational Model

[TODO: Separate the formal Lean predicates, external hardness assumptions, and runtime trust-boundary obligations for parsing, hashing, proof verification, and state transition.]

3 Mathematical Framework

3.1 Fibrational Semantics

The semantic layer is motivated by Helfer's first-order homotopical logic [Hel19]. In this paper, the corresponding Lean files are used as a formalized interface for the semantic structure needed by Homotopical Encrypt, not as a claim that every external theorem from Helfer's paper has been reproved.

3.2 Fibrational Identity Objects

Public equality is not strict equality of hidden representatives. It is mediated by public classifiers, path data, and equivalence of the observable structure. This is the semantic source of the hidden-operator model used in HCP.

3.3 Homotopy Classes

Public shadows may identify multiple hidden representatives while preserving the invariants required by a verifier. The projection framework later turns this semantic quotienting into finite public evidence.

4 Cochain Commitment Framework

4.1 Cochain Commitments

Cochain commitments provide the public handle on hidden cochain data. The Lean surface treats the canonical payload as a mathematical object with a modulus, edge values, declared edge count, and topology hash.

4.2 Commitment Map

[TODO: Describe the map from hidden cochain data into public commitments, including what is preserved and what remains hidden.]

4.3 Projection Operator

Projection creates verifier-facing public artifacts from hidden or geometric data. The projection need not be injective; it must preserve enough structure to support the relevant verification theorem.

4.4 Fundamental Properties

[TODO: State binding, well-formedness, topology-binding, and non-collapse properties. Keep byte-level hash and parser claims in the runtime/TCB boundary.]

5 Projection Framework

5.1 Projection Theorem

The staged projection chain has the following shape:

FrameWitness
→ Projective Plucker shadow
→ committed cochain/local syndrome
→ plane shadow
→ finite-field received word
→ canonical Reed–Solomon target.

Each stage has local gates and certificates. The orchestrating projection-chain theorem composes these local certificates without rechecking them.

5.2 Preserved Structure

[TODO: List stage-by-stage invariants: public binding, quotient-aware shadow identity, local plane consistency, metric calibration, and final RS well-formedness.]

5.3 Lost Structure

Projection is deliberately lossy. The theorem is not an injectivity theorem; it is a controlled public projection theorem with calibrated non-collapse and far-separation hypotheses where they are needed.

6 Relationship To CVP

CVP asks for a closest vector in an explicitly presented lattice. HCP asks for a harmonic cochain in the operative Hodge kernel, and in the Homotopical Encrypt model that kernel is not explicitly presented by public data.

6.1 Reduction Direction

[TODO: State the formalized comparison direction: a suitable Hodge/HCP solver induces a solver for the projected explicit-lattice CVP problem.]

6.2 Oracle Insufficiency

[TODO: State the hidden-operator separation: an oracle for explicit known-lattice CVP does not by itself determine the hidden operator or harmonic kernel.]

6.3 Structural Comparison

The comparison separates three surfaces:

(i) known-lattice closest-vector search;

(ii) hidden-operator HCP;

(iii) public projection/proximity verification.

7 Formal Verification

7.1 Lean Formalization

The Lean development records the mathematical predicates, reductions, projection stages, gate certificates, and trusted theorem boundaries used by the paper. The proof is not a proof of the Rust runtime; runtime parsing, canonical encodings, domain separation, and proof verification are handled by a separate TCB boundary.

7.2 Repository Structure

[TODO: Summarize the module split: Axioms, CVP/Hardness, Laplacian, Proximity, Gates, CanonicalRS, TCB, and Chain.]

7.3 Theorem Correspondence

[TODO: Insert Lean-to-paper correspondence tables for definitions and theorems, with status labels: Lean-proved, cited external theorem, runtime obligation, or assumption boundary.]

8 Reed–Solomon Projection And Proximity Corollary

8.1 Canonical Reed–Solomon Endpoint

The proximity endpoint is stated over the paper-style alphabet shape $C \subseteq (\mathbb{F}^s)^n$. The canonical Homotopical Encrypt evaluation is the ordinary Reed–Solomon slice at width $s = 1$.

8.2 Mutual Correlated Agreement

[TODO: State the MCA definition in the vocabulary of the Proximity Prize problem statement and identify which bounds are cited rather than reproved.]

8.3 Corollary

[TODO: State the verifier-facing corollary obtained by composing the projection chain with the cited Reed–Solomon MCA bounds.]

9 Future Directions

9.1 Cryptographic Protocol

[TODO: Describe the protocol layer that consumes the formal objects.]

9.2 Anonymous Identity System

[TODO: Develop the public-shadow/hidden-representative architecture as an anonymous identity mechanism.]

9.3 Sheaf Storage

[TODO: Discuss storage and retrieval systems where public commitments expose controlled sheaf/cochain projections.]

9.4 Vertra Chain

Vertra is the working name for the chain/network layer. The paper should keep the mathematical protocol name separate from product and token naming.

9.5 Complexity-Theoretic Extensions

[TODO: Refine reductions, oracle separations, and hidden-operator hardness assumptions.]

A Repository Dependency Graph

[TODO: Include the dependency graph and build targets for the theorem surfaces used in the paper.]

B Lean/Paper Correspondence

[TODO: Include tables matching Lean definitions and theorems to paper definitions and theorems.]

References

[Hel19] Adam Helfer. First-order homotopical logic. *arXiv preprint*, 2019.